

UJIAN AKHIR SEMESTER

disusun untuk memenuhi UAS
Praktikum Pembelajaran Mesin

Oleh:

DIAN ISLAMI
(2308107010048)



PROGRAM STUDI INFORMATIKA FAKULTAS
MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
DARUSSALAM, BANDA ACEH
2026

BAB 1

PENDAHULUAN

1.1. Latar Belakang

Identifikasi spesies serangga secara manual membutuhkan keahlian entomologi yang mendalam dan waktu yang tidak sedikit. Di Indonesia, dengan keanekaragaman hayati yang sangat tinggi, kemampuan untuk mengidentifikasi serangga secara cepat dan akurat memiliki nilai ekologis maupun pertanian yang besar. Serangga berperan penting dalam ekosistem, mulai dari penyerbukan tanaman, pengendalian hama alami, hingga sebagai indikator kesehatan lingkungan.

Perkembangan pesat dalam bidang computer vision dan deep learning membuka peluang untuk membangun sistem identifikasi otomatis yang dapat mengenali spesies serangga hanya dari sebuah foto. Teknik transfer learning dengan arsitektur modern seperti EfficientNet memungkinkan pelatihan model yang akurat bahkan dengan sumber daya komputasi yang terbatas, dengan memanfaatkan bobot pra-latih dari dataset berskala besar seperti ImageNet.

Di samping identifikasi visual, pemahaman mendalam mengenai suatu spesies seperti taksonomi, habitat, peran ekologis, dan fakta unik, memerlukan basis pengetahuan yang kaya. Integrasi Large Language Model (LLM) seperti Gemini 2.5 Flash memungkinkan sistem tidak hanya mengenali, tetapi juga "menjelaskan" serangga yang teridentifikasi secara naratif dan informatif kepada pengguna.

Proyek ini, yang diberi nama LensArthropoda, menggabungkan kedua kapabilitas tersebut dalam sebuah aplikasi web yang utuh: model klasifikasi berbasis deep learning untuk inferensi gambar, dan LLM untuk menghasilkan wawasan ilmiah (AI Insights), disajikan melalui antarmuka yang modern dan responsif.

1.2. Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam proyek ini adalah:

1. Bagaimana membangun model klasifikasi citra serangga yang akurat menggunakan teknik *transfer learning* dengan arsitektur EfficientNet-B3?
2. Bagaimana mengintegrasikan model ML dengan layanan Gemini API untuk menghasilkan informasi ilmiah yang kaya terhadap hasil prediksi?
3. Bagaimana menyajikan sistem identifikasi tersebut dalam antarmuka web yang responsif, elegan, dan mudah digunakan?

1.3. Tujuan

Tujuan dari proyek ini adalah:

1. Melatih dan mengeksport model klasifikasi serangga berbasis EfficientNet-B3 menggunakan dataset yang disediakan di Kaggle.
2. Membangun backend REST API menggunakan FastAPI yang menangani inferensi model dan integrasi Gemini.
3. Membangun frontend berbasis Next.js yang menampilkan hasil identifikasi beserta AI Insights dalam tampilan yang bersih dan informatif.

1.4. Manfaat

Manfaat yang didapat dari proyek ini adalah”

1. **Akademis:** Menjadi implementasi nyata dari konsep transfer learning, model serving, dan integrasi LLM dalam satu pipeline terpadu.
2. **Praktis:** Dapat digunakan oleh peneliti, petani, atau masyarakat umum untuk mengidentifikasi serangga dari foto dengan cepat tanpa keahlian khusus.
3. **Ekologis:** Meningkatkan kesadaran publik terhadap keanekaragaman dan peran ekologis serangga melalui informasi yang dihasilkan oleh AI.

BAB II

METODOLOGI DAN IMPLEMENTASI

2.1. Arsitektur Sistem

LensArthropoda menerapkan arsitektur tiga lapis (*three-tier architecture*) yang terdiri atas frontend, backend, dan layanan kecerdasan buatan. Pendekatan ini digunakan untuk memisahkan tanggung jawab setiap komponen sehingga sistem lebih mudah dikembangkan, dipelihara, dan dikembangkan lebih lanjut.

Frontend dibangun menggunakan Next.js dan Tailwind CSS sebagai antarmuka pengguna. Backend menggunakan FastAPI untuk menangani proses inferensi model serta komunikasi dengan layanan AI. Ketika pengguna mengunggah gambar serangga, frontend mengirimkan gambar ke backend melalui REST API untuk diproses.

Pada sisi backend, gambar terlebih dahulu dianalisis menggunakan model klasifikasi EfficientNet-B3 yang telah dilatih sebelumnya. Hasil prediksi kemudian digunakan sebagai masukan bagi Gemini 2.5 Flash Lite untuk menghasilkan informasi tambahan seperti taksonomi, habitat, peran ekologis, dan fakta unik. Seluruh hasil dikembalikan dalam format JSON dan ditampilkan kembali pada antarmuka pengguna.

2.2. Pelatihan Model

2.2.1. Lingkungan & Konfigurasi Awal

Pelatihan dilakukan di lingkungan Kaggle Notebooks dengan akselerator GPU T4 $\times$ 2. Library utama yang digunakan adalah PyTorch sebagai framework deep learning, timm (PyTorch Image Models) sebagai sumber arsitektur pretrained. Seluruh hyperparameter training terdapat di satu dictionary CONFIG:

Parameter	Nilai
Model	EfficientNet-B3
Ukuran Gambar	300×300
Batch Size	32
Jumlah Epoch	100
Learning Rate Head	1×10^{-3}
Learning Rate Backbone	1×10^{-5}
Weight Decay	1×10^{-4}
Label Smoothing	0,1
Patience Early Stopping	10

Scheduler	Cosine Annealing
Warmup Epochs	3
Jumlah Kelas	118

2.2.2. Eksplorasi & Analisis Dataset

Sebelum training, dilakukan eksplorasi menyeluruh terhadap dataset untuk memahami karakteristiknya. Notebook memuat folder train, val, dan test secara otomatis melalui fungsi pencarian rekursif.

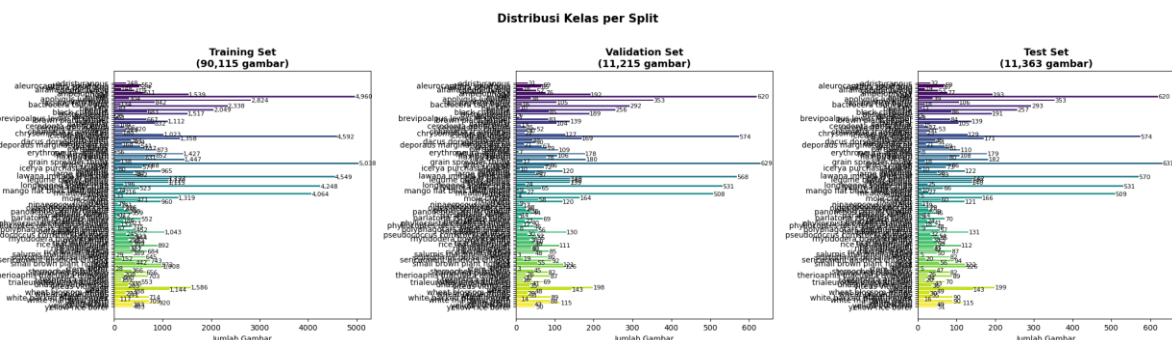
```
Struktur /kaggle/input (folder only):
├── datasets
│   ├── baxtiyorbotiraliyev
│   │   ├── insects
│   │   │   └── dataset
│   ├── dynoodel
│   ├── modelmodel
│   └── poinnpoinn

TRAIN : /kaggle/input/datasets/baxtiyorbotiraliyev/insects/dataset/train
VAL   : /kaggle/input/datasets/baxtiyorbotiraliyev/insects/dataset/val
TEST  : /kaggle/input/datasets/baxtiyorbotiraliyev/insects/dataset/test

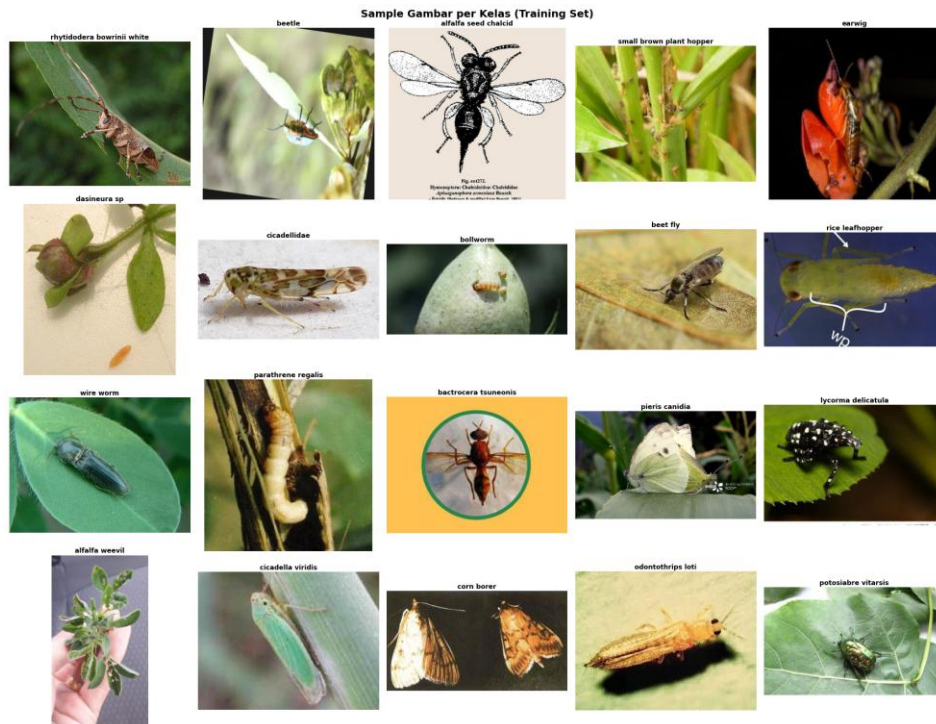
Classes dari classes.txt (118 kelas)

0. adristyrannus
1. aleurocanthus spiniferus
2. alfalfa plant bug
3. alfalfa seed chalcid
```

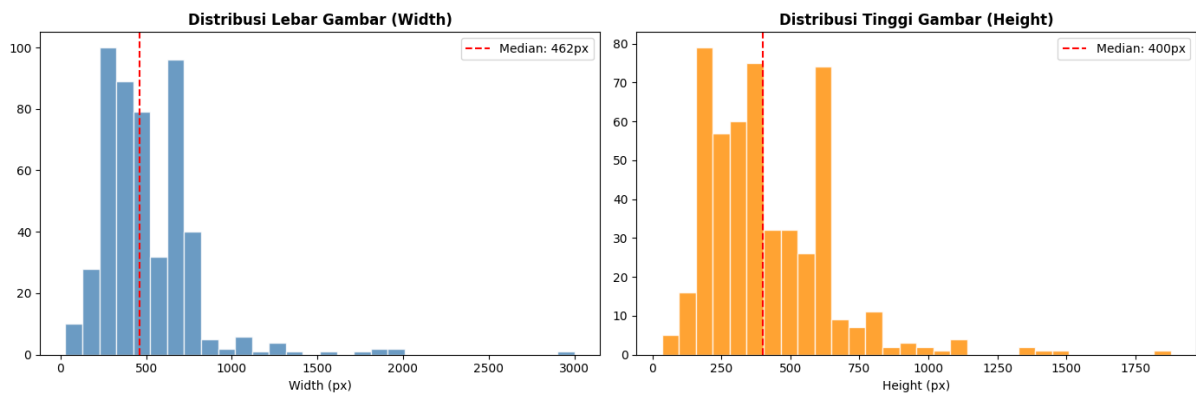
Dataset yang digunakan terdiri dari 118 kelas serangga dengan total 112.693 gambar. Data telah dibagi ke dalam tiga subset, yaitu 90.115 training, 11.215 validation, dan 11.363 testing. Setelah dilakukan analisis distribusi data, ditemukan adanya ketidakseimbangan jumlah sampel antar kelas. Pada data training, kelas sampel paling sedikit hanya 28 gambar, sedangkan kelas terbesar 5.038 gambar, menghasilkan imbalance ratio sebesar 179,93x. Untuk mengatasi permasalahan tersebut, digunakan WeightedRandomSampler pada tahap pelatihan. Teknik ini memberikan peluang sampling yang lebih besar pada kelas dengan jumlah sampel yang sedikit sehingga distribusi data lebih seimbang.



Ditampilkan juga sampel gambar dari setiap kelas, serta analisis distribusi ukuran gambar (width dan height) untuk memilih `img_size` yang sesuai dengan karakteristik dataset.



Analisis Ukuran Gambar (sampel 500 gambar)



2.2.3. Preprocessing & Augmentasi Data

Diterapkan dua jenis transformasi, yaitu data training dengan augmentasi dan data validasi/test tanpa augmentasi agar evaluasi mencerminkan performa model pada data asli. Augmentasi yang digunakan meliputi:

- **Resize (332) + RandomCrop (300)** untuk menghasilkan variasi posisi objek dalam gambar.
- **RandomHorizontalFlip (p=0,5)** untuk meningkatkan keragaman data melalui pembalikan horizontal.
- **RandomVerticalFlip (p=0,2)** untuk mensimulasikan variasi orientasi gambar yang masih relevan.
- **RandomRotation (30°)** untuk mensimulasikan perbedaan sudut pengambilan gambar.

- **ColorJitter** untuk merepresentasikan variasi kondisi pencahayaan, kontras, saturasi, dan warna.
- **RandomGrayscale (p=0,05)** agar model tidak hanya bergantung pada informasi warna, tetapi juga tekstur dan bentuk.
- **RandomPerspective** untuk mensimulasikan distorsi perspektif akibat sudut pengambilan gambar yang berbeda.
- **RandomErasing (p=0,2)** untuk mensimulasikan oklusi sebagian objek dan meningkatkan kemampuan generalisasi model.
- **Normalisasi ImageNet** menggunakan nilai mean dan standar deviasi yang sama dengan data pelatihan EfficientNet-B3 agar distribusi input konsisten dengan bobot pretrained.

2.2.4. Penanganan Gambar Rusak

Untuk meningkatkan ketahanan proses pelatihan, digunakan kelas RobustImageFolder yang memodifikasi perilaku ImageFolder bawaan PyTorch. Jika ditemukan gambar yang rusak atau tidak dapat dibuka, dataset akan mencoba mengambil sampel lain secara otomatis. Selain itu, seluruh gambar dikonversi ke format RGB agar memiliki jumlah kanal yang konsisten selama proses training.

```
1 class RobustImageFolder(Dataset):
2     def __init__(self, root, transform=None):
3         self.base = ImageFolder(root, transform=None)
4         self.transform = transform
5         self.classes = self.base.classes
6         self.class_to_idx = self.base.class_to_idx
7         self.samples = self.base.samples
8         print(f' {root.name} -> {len(self.samples):,} file ({len(self.classes)} kelas):'
9       ...
10
```

2.2.5. DataLoader & WeightedRandomSampler

Data pelatihan dimuat menggunakan DataLoader dengan WeightedRandomSampler untuk mengatasi ketidakseimbangan jumlah sampel antar kelas. Teknik ini memberikan peluang sampling yang lebih besar pada kelas minoritas sehingga distribusi data yang diterima model menjadi lebih seimbang. Selain itu, digunakan konfigurasi pin_memory, num_workers, dan persistent_workers untuk meningkatkan efisiensi proses loading data.

```
1 def make_weighted_sampler(dataset):
2     labels = [label for _, label in dataset.samples]
3     class_counts = Counter(labels)
4     class_weights = {cls: 1.0 / count for cls, count in class_counts.items()}
5     sample_weights = [class_weights[label] for label in labels]
6     sampler = WeightedRandomSampler(
7         weights=sample_weights,
8         num_samples=len(sample_weights),
9         replacement=True
10    )
```

2.2.6. Arsitektur Model: InsectClassifier

Model klasifikasi dibangun menggunakan pendekatan *transfer learning* dengan EfficientNet-B3 sebagai backbone pretrained. Lapisan classifier bawaan dihapus dan diganti

dengan *custom classifier head* yang terdiri dari Batch Normalization, Dropout, Linear Layer, dan aktivasi SiLU. Arsitektur ini memanfaatkan kemampuan ekstraksi fitur EfficientNet-B3 sekaligus meningkatkan kemampuan adaptasi model terhadap tugas klasifikasi serangga.

```
1 class InsectClassifier(nn.Module):
2     def __init__(self, model_name: str, num_classes: int, dropout_rate: float = 0.4):
3         super().__init__()
4         self.backbone = timm.create_model(
5             model_name,
6             pretrained=True,
7             num_classes=num_classes,
8             global_pool='avg'
9         )
10         ...
```

2.2.7. Sistem Checkpoint (Resume Otomatis)

Sistem checkpoint digunakan untuk menyimpan progres pelatihan secara berkala sehingga proses training dapat dilanjutkan apabila terjadi gangguan atau sesi komputasi berakhir. Checkpoint mencakup bobot model, status optimizer, scheduler, dan riwayat pelatihan. Untuk menghemat ruang penyimpanan, hanya dua checkpoint terbaru yang dipertahankan.

```
1 def save_checkpoint(epoch, phase, model, optimizer, scheduler, best_val_loss, best_val_acc, history, scalar_name):
2     state_dict = model.module.state_dict() if hasattr(model, 'module') else model.state_dict()
3     path = f'{CONFIG["checkpoint_dir"]}/ckpt_epoch_{epoch}.pth'
4     payload = {
5         'epoch': epoch,
6         'phase': phase,
7         'model_state': state_dict,
8         'optimizer_state': optimizer.state_dict(),
9         'scheduler_state': scheduler.state_dict(),
10        'best_val_loss': best_val_loss,
11        'best_val_acc': best_val_acc,
12        'history': history,
13    }
14     ...
```

2.2.8. Training Dua Fase (Phase 1 & Phase 2)

Training menggunakan strategi **two-phase fine-tuning** yang merupakan praktik terbaik dalam transfer learning:

1. Phase 1 - Melatih Classifier Head (15 epoch): Pada fase pertama, seluruh parameter backbone EfficientNet-B3 dibekukan (*freeze*) sehingga hanya classifier head yang diperbarui selama proses pelatihan. Pendekatan ini bertujuan untuk menyesuaikan classifier terhadap fitur-fitur yang telah dipelajari model pretrained tanpa mengubah representasi fitur pada backbone. Konfigurasi yang digunakan:

- Optimizer: AdamW ($lr = 1 \times 10^{-3}$, $weight_decay = 1 \times 10^{-4}$)
- Loss Function: CrossEntropyLoss dengan `label_smoothing = 0,1`
- Scheduler: Cosine Annealing Learning Rate
- Mixup Augmentation: Tidak digunakan
- Automatic Mixed Precision (AMP): Aktif
- Checkpoint: Disimpan setiap epoch untuk mendukung auto-resume training
- Durasi: 15 epoch

2. Phase 2 - Fine-Tuning Seluruh Model: Setelah classifier head mencapai konvergensi awal, seluruh parameter backbone dibuka kembali (*unfreeze*) sehingga proses pelatihan dilakukan secara end-to-end. Pada fase ini diterapkan *differential learning rate*, yaitu penggunaan learning rate yang berbeda untuk backbone dan classifier. Konfigurasi:

- Optimizer: AdamW
- Learning Rate Backbone: 1×10^{-5}
- Learning Rate Classifier: 1×10^{-4}
- Weight Decay: 1×10^{-4}
- Scheduler: Cosine Annealing Warm Restarts
- Mixup Augmentation: Aktif ($\alpha = 0,4$)
- Automatic Mixed Precision (AMP): Aktif
- Gradient Clipping: $\text{max_norm} = 1,0$
- Early Stopping: $\text{patience} = 10$
- Checkpoint: Disimpan setiap epoch untuk mendukung auto-resume training
- Best Model Saving: Model terbaik disimpan berdasarkan performa validasi

Selain strategi two-phase fine-tuning, pelatihan juga memanfaatkan Automatic Mixed Precision (AMP) untuk meningkatkan efisiensi penggunaan GPU, Mixup Augmentation sebagai teknik regularisasi tambahan, Early Stopping untuk mencegah overfitting, serta CrossEntropyLoss dengan Label Smoothing sebesar 0,1 untuk meningkatkan kemampuan generalisasi model.

2.2.9. Ekspor Model

Model yang telah selesai dilatih diekspor ke format TorchScript menggunakan `torch.jit.trace()` untuk memudahkan proses deployment. Selain itu, metadata seperti nama kelas, ukuran citra, dan parameter normalisasi disimpan dalam berkas JSON agar dapat digunakan kembali saat proses inferensi pada sistem backend.

```
Output files:
 46.77 MB /kaggle/working/best_model.pth
  0.00 MB /kaggle/working/checkpoints
  0.61 MB /kaggle/working/class_distribution.png
  0.01 MB /kaggle/working/classification_report.txt
  4.06 MB /kaggle/working/confusion_matrix.png
  0.07 MB /kaggle/working/image_size_distribution.png
139.46 MB /kaggle/working/latest_checkpoint.pth
  0.01 MB /kaggle/working/model_metadata.json
 47.62 MB /kaggle/working/model_traced.pt
  0.71 MB /kaggle/working/per_class_f1.png
  4.85 MB /kaggle/working/sample_images.png
  0.17 MB /kaggle/working/training_curves.png
```

2.3. Pengembangan Backend

2.3.1. Modul Inferensi: `ml_service.py`

Kelas `InsectClassifier` bertanggung jawab untuk memuat model `EfficientNet-B3` yang telah diekspor ke format `TorchScript` dan melakukan proses inferensi gambar. Fitur utama modul ini meliputi:

- Memuat model `TorchScript` menggunakan `torch.jit.load()`.
- Melakukan preprocessing gambar berupa konversi RGB, resize, dan normalisasi menggunakan parameter `ImageNet`.
- Menghasilkan tiga prediksi terbaik (*top-3 prediction*) beserta probabilitasnya menggunakan `softmax` dan `topk`.
- Mendukung eksekusi otomatis pada CPU maupun GPU melalui penggunaan `map_location`.

2.3.2. Modul AI Insights: `gemini_service.py`

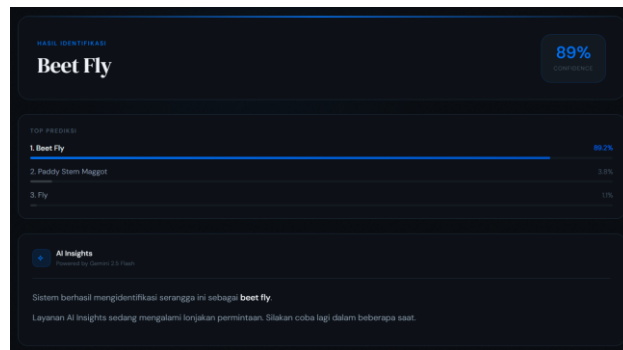
Kelas `GeminiExpert` mengintegrasikan `Gemini 2.5 Flash Lite` untuk menghasilkan informasi tambahan berdasarkan hasil klasifikasi model. Fitur utama modul ini meliputi:

- Menggunakan `ThinkingConfig` dengan `thinking_budget=512` untuk meningkatkan kualitas respons.
- Menghasilkan informasi mengenai identifikasi, taksonomi, habitat, peran ekologis, dan fakta unik serangga.
- Menggunakan *streaming response* untuk mengurangi latensi respons.
- Menghasilkan keluaran format `Markdown` sehingga mudah ditampilkan pada frontend.
- Memanfaatkan `Google Search grounding` untuk mendukung informasi yang lebih relevan dan mutakhir.

2.3.3. Entry Point & Fallback Mechanism: `main.py`

File `main.py` berfungsi sebagai pusat koordinasi yang menghubungkan frontend, model machine learning, dan layanan Gemini. Fitur utama modul ini meliputi:

- Endpoint `POST /analyze` untuk menerima gambar dari pengguna.
- Menjalankan inferensi model dan menghasilkan prediksi spesies serangga.
- Mengirim hasil prediksi utama ke Gemini untuk memperoleh AI Insights.
- Mengembalikan hasil klasifikasi dan AI Insights dalam satu respons `JSON`.
- Menerapkan *fallback mechanism* sehingga hasil prediksi tetap dapat ditampilkan jika Gemini mengalami gangguan.



- Menggunakan konfigurasi CORS untuk mendukung komunikasi antara backend dan frontend selama pengembangan.
- *Exception handling* mengembalikan respons HTTP 500 jika terjadi kesalahan server.

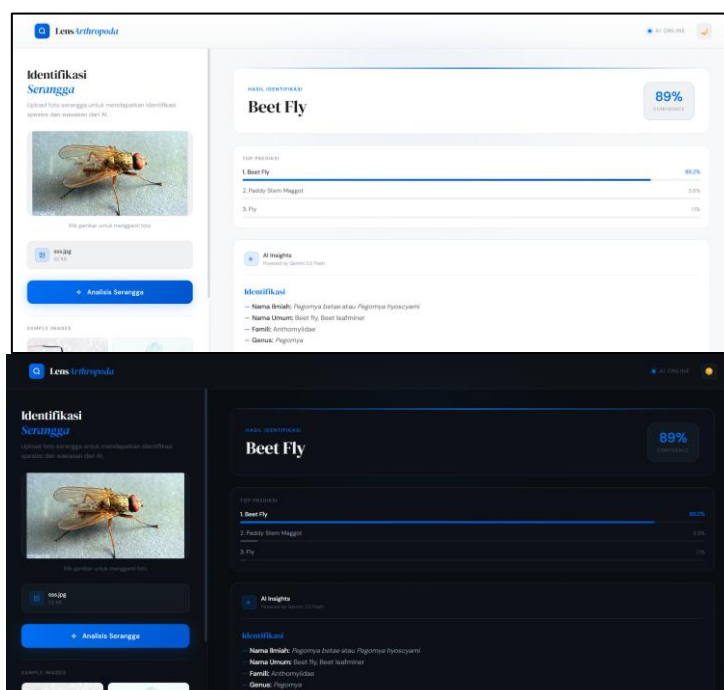
Code	Details
400	Error: Bad Request
<i>Undocumented</i>	Response body
	<pre>{ "detail": "File harus berupa gambar"}</pre>

2.4. Pengembangan Frontend

Frontend dibangun menggunakan Next.js 14 (App Router) dan Tailwind CSS sebagai *utility-first CSS framework*, serta tipografi *DM Sans* dan *DM Serif Display* dari Google Fonts.

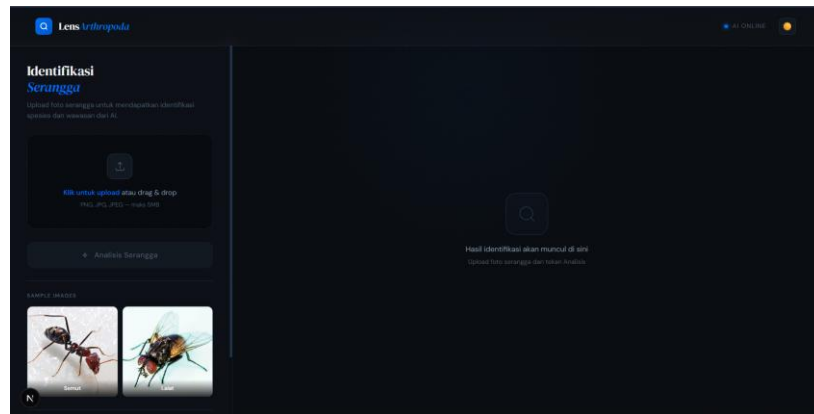
2.4.1. Desain Visual

Sistem menggunakan kombinasi warna gelap (dark mode) dan terang (light mode) yang dapat diubah oleh pengguna. Pengaturan warna, tipografi, serta elemen visual disimpan dalam CSS Variables sehingga mudah dikelola dan konsisten di seluruh aplikasi.

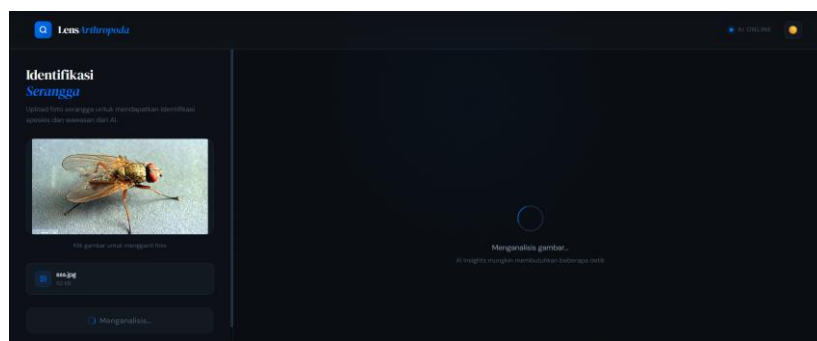


2.4.2. Implementasi Halaman Utama

Halaman utama dibangun menggunakan React Functional Component dan berfungsi sebagai pusat interaksi pengguna. Halaman ini terdiri dari dua bagian utama, yaitu panel unggah gambar di sebelah kiri dan panel hasil identifikasi di sebelah kanan. Pengguna memulai identifikasi dengan mengunggah gambar serangga melalui fitur upload atau drag-and-drop.



Setelah gambar dipilih, sistem menampilkan pratinjau gambar dan mengirimkannya ke backend untuk dianalisis. Selama proses berlangsung, frontend menampilkan indikator loading sebagai umpan balik visual kepada pengguna.



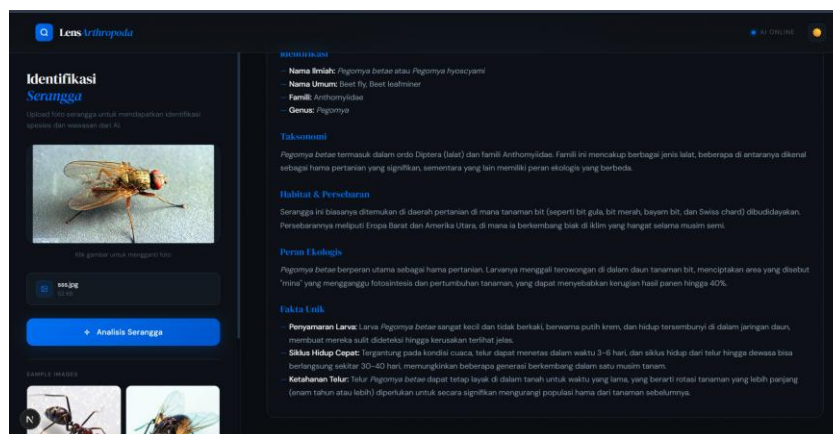
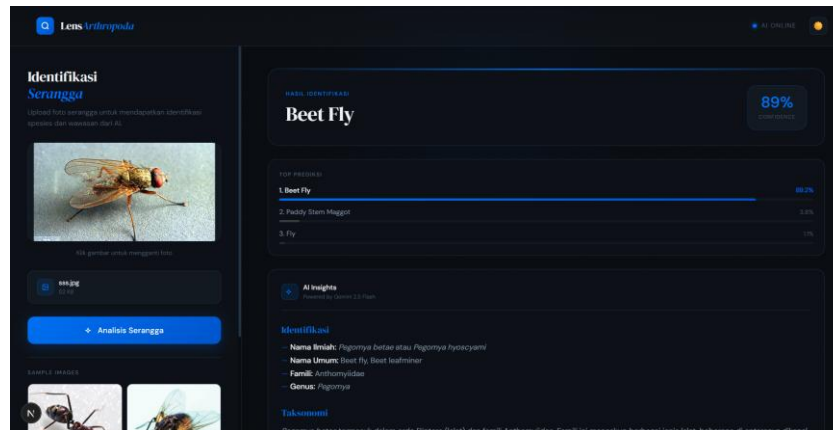
Setelah analisis selesai, hasil identifikasi, tingkat kepercayaan model, serta AI Insights ditampilkan pada halaman yang sama sehingga pengguna dapat langsung melihat informasi yang dihasilkan tanpa perlu berpindah halaman.

2.4.3. Implementasi Hasil ResultCard.tsx

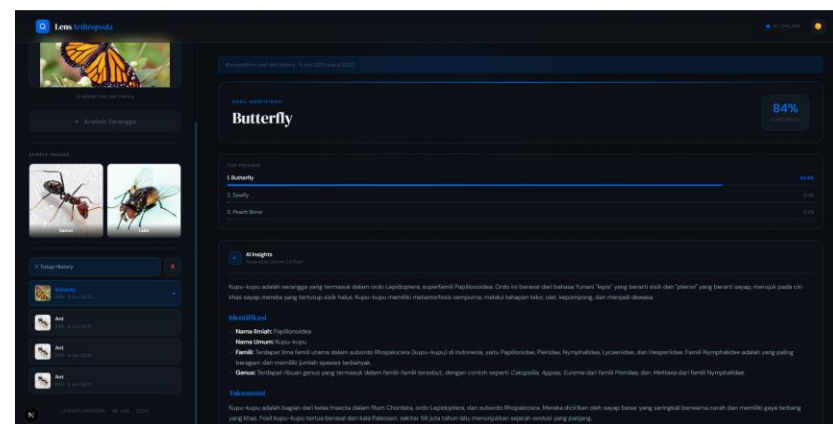
Komponen ini menerima respons JSON dari backend dan menampilkannya dalam tata letak yang terstruktur:

- **Header Identifikasi:** Menampilkan nama spesies utama dengan tipografi serif dan skor kepercayaan (*confidence*) dengan kode warna dinamis, biru untuk $\geq 75\%$, kuning untuk $\geq 50\%$, merah untuk di bawahnya.

- **Top Prediksi:** Menampilkan tiga prediksi teratas dengan progress bar animasi yang lebarnya proporsional terhadap probabilitas masing-masing kelas.
- **AI Insights:** Teks Markdown dari Gemini dirender menggunakan react-markdown dengan custom styling melalui kelas .ai-prose, menghasilkan tampilan heading, bold text, dan daftar yang konsisten dengan tema aplikasi.



- **Riwayat Analisis (History):** Hasil identifikasi disimpan secara lokal menggunakan localStorage, mencakup nama spesies, tingkat kepercayaan, waktu analisis, thumbnail gambar, serta detail hasil prediksi. Pengguna dapat membuka kembali hasil analisis sebelumnya melalui panel *History* tanpa perlu melakukan proses inferensi ulang.



BAB III

HASIL DAN PEMBAHASAN

3.1. Gambaran Umum Eksperimen

Penelitian ini menggunakan model EfficientNet-B3 untuk klasifikasi citra serangga dan hama pertanian dengan 118 kelas. Model dilatih menggunakan platform Kaggle dengan dukungan dual GPU melalui mekanisme DataParallel. Dataset terdiri dari ribuan citra yang dibagi ke dalam split training, validasi, dan testing. Konfigurasi pelatihan menggunakan ukuran gambar 300×300 piksel, batch size 32, dan normalisasi berdasarkan mean dan standar deviasi ImageNet ([0.485, 0.456, 0.406] dan [0.229, 0.224, 0.225]).

3.2. Konfigurasi dan Arsitektur Model

Model yang digunakan adalah EfficientNet-B3 yang diinisialisasi dengan bobot pre-trained ImageNet. Arsitektur ini dipilih karena keunggulannya dalam efisiensi komputasi melalui compound scaling yang menyeimbangkan kedalaman, lebar, dan resolusi jaringan.

Parameter	Nilai
Arsitektur	EfficientNet-B3
Jumlah Kelas	118
Ukuran Input	300 × 300 piksel
Total Parameter	11.547.806
Parameter Trainable	11.547.806 (100%)
Batch Size	32
Num Workers	4
Learning Rate (Head)	0.001
Learning Rate (Backbone)	0.00001
Weight Decay	0.0001
Label Smoothing	0.1
Scheduler	Cosine Annealing
Warmup Epochs	3
Patience (Early Stopping)	10
Min Delta	0.0001
Max Epochs	100

3.3. Kurva Pelatihan (Training Curves)



3.3.1. Loss Training dan Validasi

Pada fase awal (epoch 1-15), training loss turun secara signifikan dari sekitar 3.5 menuju 2.0, yang mengindikasikan model berhasil mempelajari fitur-fitur diskriminatif dari dataset secara cepat. Fase ini merupakan fase warm-up dan pemanasan di mana hanya kepala classifier yang dilatih secara agresif.

Pada sekitar epoch 25 terdapat lonjakan loss yang drastis (spike) di mana training loss melonjak hingga mendekati nilai 9.0. Fenomena ini secara langsung berkorelasi dengan transisi ke Phase 2 (ditandai garis putus-putus), yaitu saat seluruh backbone mulai di-unfreeze dan dilatih bersama dengan learning rate backbone yang sangat kecil ($1e-5$). Spike ini terjadi karena optimizer di-reset atau learning rate scheduler dimulai ulang, menyebabkan gradien yang lebih besar dari backbone yang baru saja di-unlock menginterferensi representasi yang sudah terbentuk. Meskipun demikian, model berhasil pulih dalam beberapa epoch berikutnya.

Setelah epoch 30 hingga 100, loss training dan validasi keduanya turun secara stabil dan konvergen, mendekati nilai sekitar 1.5, menandakan model telah memasuki fase pembelajaran yang stabil dengan generalisasi yang baik.

3.3.2. Akurasi Training dan Validasi

Akurasi validasi secara konsisten **lebih tinggi dari akurasi training** sepanjang sebagian besar proses pelatihan (mencapai ~80% vs ~69% di akhir). Fenomena ini disebut *generalization gap* yang terbalik, dan pada konteks ini sangat mungkin disebabkan oleh beberapa faktor:

1. Augmentasi data hanya diterapkan pada set training, sehingga training lebih sulit secara artifisial.
2. Label smoothing mengurangi nilai loss training lebih dari validasi, sehingga model yang dievaluasi tanpa smoothing (validasi) tampak lebih akurat.
3. Dropout aktif selama training namun dinonaktifkan saat evaluasi validasi.

3.3.3. Learning Rate Schedule

Grafik learning rate menunjukkan pola Cosine Annealing yang diaplikasikan berulang. Pada Phase 1, learning rate kepala turun dari $1e-3$ secara kosinusoidal. Pada Phase 2 terdapat tiga siklus cosine annealing terpisah yang terlihat jelas di grafik. Siklus ini dikenal sebagai SGDR (Stochastic Gradient Descent with Warm Restarts).

3.4. Hasil Evaluasi Keseluruhan

Metrik	Nilai
Test Accuracy (Top-1)	80.15%
Top-3 Accuracy	92.28%
Top-5 Accuracy	94.73%
Macro Average Precision	71.02%
Macro Average Recall	75.32%
Macro Average F1-Score	72.24%
Weighted Average Precision	82.03%
Weighted Average Recall	80.15%
Weighted Average F1-Score	80.46%

Akurasi top-1 sebesar 80.15% pada 118 kelas merupakan hasil yang sangat kompetitif mengingat kompleksitas tugas klasifikasi multikelas yang tinggi. Lebih signifikan lagi, top-3 accuracy mencapai 92.28% dan top-5 accuracy mencapai 94.73%, yang berarti dalam 9 dari 10 kasus, kelas yang benar ada di antara 3 prediksi teratas model. Ini sangat relevan untuk aplikasi pertanian praktis, di mana pengguna dapat memilih dari beberapa kandidat prediksi.

Selisih antara macro average F1 (72.24%) dan weighted average F1 (80.46%) sebesar 8 poin mengindikasikan adanya ketidakseimbangan kinerja antar kelas, dimana model bekerja lebih baik pada kelas dengan jumlah sampel banyak dibanding kelas dengan sampel sedikit. Hal ini lumrah mengingat distribusi kelas yang tidak seimbang dalam dataset serangga.

3.5. Analisis Per-Kelas

3.5.1. Kelas dengan Performa Terbaik ($F1\text{-Score} \geq 0.85$)

Kelas	Precision	Recall	F1-Score	Support
Snail	1.0000	1.0000	1.0000	122
Dragonfly	0.9718	1.0000	0.9857	69
Ladybug	0.9683	1.0000	0.9839	122

Spider	0.9692	1.0000	0.9844	126
Bee	0.9766	0.9932	0.9848	294
Slug	0.9592	1.0000	0.9792	94
Mole Cricket	0.9704	0.9880	0.9791	166
Leaf Beetle	0.9772	0.9806	0.9789	567
Mosquito	0.9375	1.0000	0.9677	60

Kelas-kelas dengan F1-Score sempurna atau mendekati sempurna umumnya adalah organisme yang memiliki morfologi yang khas dan mudah dibedakan secara visual: siput (snail) dengan cangkangnya, capung (dragonfly) dengan sayap transparan dan tubuh memanjang, laba-laba (spider), dan kepik (ladybug) dengan pola merah-hitam yang ikonik.

3.5.2. Kelas dengan Performa Menengah (F1-Score 0.70 - 0.84)

Kelas	Precision	Recall	F1-Score	Support
Grasshopper	0.9710	0.8479	0.9052	631
Cicadellidae	0.9287	0.7944	0.8563	574
Butterfly	0.9645	0.9784	0.9714	139
Aphids	0.9383	0.7113	0.8092	620
Miridae	0.8987	0.6974	0.7854	509
Blister Beetle	0.7151	0.6963	0.7056	191
Corn Borer	0.6839	0.7719	0.7253	171

Kelas seperti aphids menunjukkan presisi tinggi (0.9383) namun recall rendah (0.7113), artinya model yakin ketika memprediksi aphids, tetapi cukup sering melewati beberapa contoh aphids yang mungkin mirip dengan kelas lain. Pola sebaliknya terlihat pada cicadella viridis (presisi 0.6369, recall 0.8837), model sering memprediksi kelas ini bahkan saat sebenarnya bukan, menandakan visual yang ambigu dengan kelas lain.

3.5.3. Kelas dengan Performa Rendah (F1-Score < 0.50)

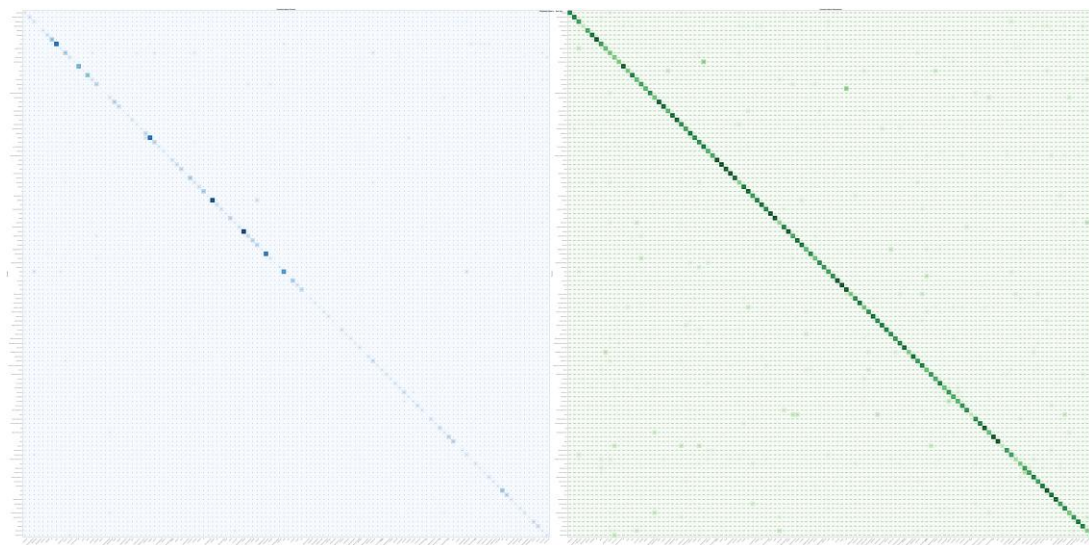
Kelas	Precision	Recall	F1-Score	Support
Stem Borer	0.3333	0.2000	0.2500	5
Sawfly	1.0000	0.2000	0.3333	5
Therioaphis maculata Buckton	0.2581	0.3077	0.2807	26
Alfalfa Seed Chalcid	0.4000	0.2105	0.2759	19
Beet Fly	0.2632	0.4545	0.3333	11

Thrips	0.3832	0.4607	0.4184	89
Flax Budworm	0.4412	0.4167	0.4286	108
Large Cutworm	0.3878	0.3800	0.3838	50
Yellow Cutworm	0.3279	0.4082	0.3636	49

Kelas-kelas dengan performa rendah ini dapat dijelaskan melalui dua faktor utama:

1. **Jumlah sampel sangat sedikit:** Stem borer dan sawfly hanya memiliki 5 sampel test, sehingga satu kesalahan saja menurunkan metrik secara dramatis. Ini bukan indikasi model buruk, melainkan keterbatasan statistik dari support yang kecil.
2. **Kemiripan visual antar kelas:** Kelas seperti thrips, flax budworm, large cutworm, dan yellow cutworm secara visual sangat mirip satu sama lain, serangga berukuran kecil dengan warna kecoklatan atau kehijauan, tanpa fitur pembeda yang mencolok. Bahkan ahli entomologi sekalipun terkadang membutuhkan pemeriksaan mikroskopik untuk membedakannya. Kesulitan ini tercermin dalam rendahnya presisi dan recall model.

3.6. Analisis Confusion Matrix



3.6.1. Pola Diagonal yang Dominan

Pada kedua versi confusion matrix (count dan normalized), diagonal utama mendominasi secara visual, yang mengkonfirmasi bahwa mayoritas prediksi model adalah benar. Pola ini konsisten dengan akurasi test 80.15%.

3.6.2. Pola Kesalahan yang Teridentifikasi

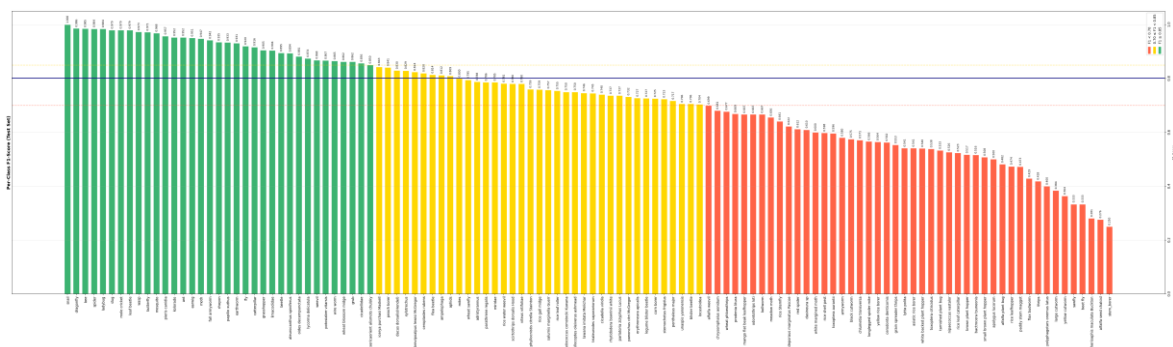
Dari confusion matrix (versi normalized, panel kanan), beberapa pola off-diagonal yang terlihat mengindikasikan kebingungan sistematis antar kelas tertentu:

- Kelas-kelas dalam famili yang sama saling membingungkan satu sama lain. Ini masuk akal secara biologis karena mereka adalah larva dari ngengat berbeda dalam famili yang sama dengan penampilan yang hampir identik di tahap tertentu.
- Kelompok leafhopper (Cicadellidae, Cicadella viridis, rice leafhopper, small brown plant hopper) menunjukkan beberapa konfusi silang karena morfologi yang serupa.
- Berbagai jenis thrips (grain spreader thrips, scirtothrips dorsalis, wheat phloeothrips, odontothrips loti) juga saling terkait dalam pola error, mengingat ukurannya yang sangat kecil dan bentuk yang hampir identik.

3.6.3. Kelas yang Tidak Pernah Salah

Beberapa kelas seperti **snail**, **spider**, **dragonfly**, **ladybug**, dan **mosquito** menunjukkan diagonal yang nyaris sempurna tanpa off-diagonal yang signifikan. Ini menunjukkan bahwa fitur visual unik kelas-kelas ini sangat mudah dipelajari oleh CNN.

3.7. Analisis Berdasarkan Grafik Per-Kelas F1-Score



Dari gambar diatas, distribusi performa model dapat dikategorikan sebagai berikut:

- **$F1 \geq 0.85$ (hijau):** Sekitar 35 kelas, umumnya serangga dengan fitur visual yang sangat distinktif atau yang memiliki jumlah data training yang memadai.
- **$F1$ antara $0.70 - 0.85$ (kuning):** Sekitar 37 kelas, performa menengah dengan beberapa ambiguitas visual atau ketidakseimbangan precision-recall.
- **$F1 < 0.70$ (merah):** Sekitar 46 kelas, serangga kecil, morfologi mirip, data sedikit.

Grafik ini dengan jelas menunjukkan bahwa **distribusi performa tidak merata** ada kelas-kelas sulit yang menarik performa makro rata-rata (72.24%) jauh di bawah performa weighted rata-rata (80.46%). Ini adalah tantangan inheren dalam klasifikasi skala besar.

3.8. Analisis Overfitting dan Underfitting

Salah satu aspek krusial dalam evaluasi model adalah menentukan apakah model mengalami overfitting, underfitting, atau berada pada kondisi generalisasi yang baik.

Berdasarkan kurva pelatihan, model ini menunjukkan indikasi underfitting parsial, bukan overfitting. Bukti-bukti yang mendukung kesimpulan ini adalah sebagai berikut:

- **Training accuracy ($\pm 69\%$) secara konsisten lebih rendah dari validation accuracy ($\pm 80\%$).** Kesenjangan terbalik ini justru merupakan ciri khas underfitting parsial yang dipicu oleh regularisasi yang kuat.
- **Training loss di akhir epoch masih berada di sekitar 1.5 dan belum benar-benar mendatar (plateau).** Jika model sudah mencapai kapasitas optimalnya, loss training seharusnya sudah flat. Kondisi ini mengindikasikan masih ada ruang bagi model untuk belajar lebih banyak dari data training jika hambatan regularisasi dikurangi.
- **Validation loss mengikuti training loss secara konsisten tanpa divergensi.** Tidak ada titik di mana validation loss mulai naik sementara training loss turun, ini mengkonfirmasi bahwa overfitting tidak terjadi.

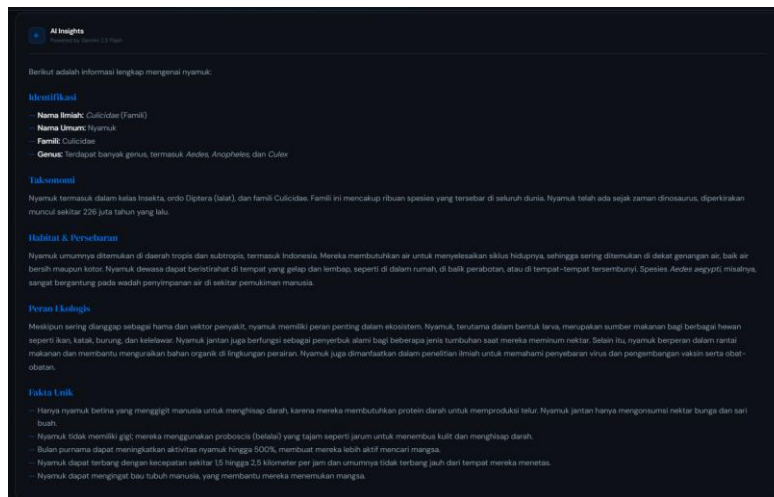
Namun penting untuk dipahami bahwa underfitting parsial ini adalah efek yang disengaja dari pilihan strategi regularisasi, bukan akibat kapasitas model yang kurang. Ada tiga mekanisme utama yang menekan training accuracy secara artifisial:

- Augmentasi data agresif selama training membuat gambar training terlihat berbeda di setiap epoch, sehingga jauh lebih sulit dibanding validasi yang tidak diaugmentasi.
- Label smoothing sebesar 0.1 mendistribusikan sebagian probabilitas dari kelas benar ke kelas lain, sehingga model tidak pernah mendapat "kredit penuh" atas prediksi yang benar selama training, menekan training accuracy ke bawah.
- Dropout aktif selama training namun dimatikan saat evaluasi validasi, yang dimana membuat model lebih lemah saat prediksi training dibanding saat prediksi validasi.

3.9. Analisis Integrasi dan Respons AI Insights (Gemini 2.5 Flash)

Selain output klasifikasi dari model EfficientNet-B3, sistem juga mengintegrasikan LLM Gemini 2.5 Flash untuk memberikan wawasan mendalam mengenai serangga yang teridentifikasi. Respons Gemini secara konsisten terstruktur dalam segmen informasi yaitu:

- Identifikasi: nama ilmiah, nama umum, famili, dan genus.
- Taksonomi: penjelasan posisi serangga dalam hierarki klasifikasi biologis beserta konteks evolusinya.
- Habitat & Persebaran: deskripsi lingkungan hidup dan distribusi geografis spesies.
- Peran Ekologis: fungsi serangga dalam ekosistem, baik sebagai dekomposer, predator, penyerbuk, maupun vektor penyakit.
- Fakta Unik: poin-poin menarik yang meningkatkan nilai edukatif aplikasi.



Integrasi Gemini 2.5 Flash menghasilkan respons yang terstruktur dan akurat secara ilmiah pada pengujian semut (confidence 83.9%) dan nyamuk (confidence 88.1%). Gemini berhasil prediksi model dengan informasi taksonomi, habitat, peran ekologis, dan fakta unik yang relevan, bahkan menyesuaikan konteks lokal Indonesia pada penjelasan nyamuk. Format output yang konsisten di seluruh pengujian mengindikasikan bahwa prompt yang diterapkan berhasil mengarahkan LLM untuk menghasilkan respons yang dapat diprediksi strukturnya.

3.10. Keterbatasan dan Analisis Kegagalan

3.10.1. Keterbatasan Data

Kelas-kelas dengan support sangat kecil (< 15 sampel test seperti bollworm=5, sawfly=5, stem_borer=5, mites=5) tidak dapat dievaluasi secara statistik yang reliable. Performa yang rendah pada kelas-kelas ini tidak serta-merta mencerminkan kelemahan arsitektur, melainkan keterbatasan data ground truth dalam test set.

3.10.2. Potensi Perbaikan

Berdasarkan analisis di atas, beberapa arah perbaikan yang dapat dilakukan antara lain:

1. Augmentasi data yang lebih agresif untuk kelas dengan sampel sedikit, termasuk teknik synthetic data generation seperti Mixup atau CutMix.
2. Focal Loss atau Class-Balanced Loss untuk menangani ketidakseimbangan kelas lebih efektif dibanding label smoothing saja.
3. Hierarki klasifikasi, memanfaatkan struktur taksonomi (famili $\rightarrow$ genus $\rightarrow$ spesies) sebagai informasi tambahan dalam loss function.
4. Ensemble model dengan arsitektur berbeda (EfficientNet-B4/B5, ConvNeXt, atau Vision Transformer) untuk meningkatkan performa terutama pada kelas sulit.
5. Test-Time Augmentation (TTA) untuk meningkatkan akurasi prediksi pada inferensi tanpa perlu pelatihan ulang.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Proyek LensArthropoda berhasil membangun sistem identifikasi serangga otomatis yang terintegrasi dengan Large Language Model secara end-to-end. Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Model Klasifikasi** Model EfficientNet-B3 yang dilatih dengan strategi two-phase fine-tuning berhasil mencapai akurasi test top-1 sebesar 80,15% dan top-3 sebesar 92,28% pada dataset 118 kelas serangga dengan total 112.693 gambar. Hasil ini tergolong kompetitif mengingat tingginya kompleksitas tugas klasifikasi multikelas dengan distribusi data yang sangat tidak seimbang (imbalance ratio 179,93x).
2. **Efektivitas Strategi Pelatihan** Kombinasi teknik WeightedRandomSampler, augmentasi data agresif, label smoothing, dropout, Mixup Augmentation, dan Cosine Annealing terbukti efektif dalam mengendalikan overfitting. Model menunjukkan kondisi underfitting parsial yang disengaja sebagai dampak dari regularisasi kuat, bukan akibat kapasitas model yang tidak memadai. Tidak ditemukan tanda-tanda overfitting sepanjang proses pelatihan.
3. **Performa Per-Kelas** Kelas dengan morfologi visual yang khas dan distinktif seperti snail, dragonfly, ladybug, dan spider mencapai F1-Score mendekati atau sama dengan 1,0. Sebaliknya, kelas dengan kemiripan visual tinggi seperti thrips, cutworm, dan budworm serta kelas dengan jumlah sampel yang sangat sedikit menunjukkan performa yang rendah. Hal ini merupakan tantangan inheren yang juga dihadapi para ahli entomologi dalam praktik nyata.
4. **Integrasi LLM** Integrasi Gemini 2.5 Flash Lite berhasil memperkaya output sistem melampaui sekadar label prediksi. Respons yang dihasilkan konsisten terstruktur mencakup identifikasi ilmiah, taksonomi, habitat, peran ekologis, dan fakta unik, serta mampu menyesuaikan konteks lokal Indonesia. Hal ini secara signifikan meningkatkan nilai edukatif dan kepraktisan aplikasi bagi pengguna non-ahli.
5. **Arsitektur Sistem** Arsitektur tiga lapis yang memisahkan frontend (Next.js), backend (FastAPI), dan layanan AI terbukti modular dan dapat dikembangkan lebih lanjut. Fallback mechanism yang diterapkan pada backend memastikan sistem tetap dapat menampilkan

hasil klasifikasi meskipun layanan Gemini mengalami gangguan, sehingga meningkatkan ketahanan sistem secara keseluruhan

4.2. Saran

Berdasarkan hasil dan keterbatasan yang telah diidentifikasi, beberapa saran pengembangan lanjutan diusulkan sebagai berikut:

1. **Peningkatan Kualitas Data:** Mengumpulkan data tambahan untuk kelas dengan sampel sangat sedikit, termasuk memanfaatkan teknik synthetic data generation seperti GAN atau Stable Diffusion untuk spesies yang sulit didokumentasikan.
2. **Arsitektur Lanjutan:** Mengeksplorasi arsitektur modern seperti Vision Transformer (ViT) atau ConvNeXt-V2, serta pendekatan ensemble model untuk meningkatkan performa pada kelas-kelas yang sulit.
3. **Pemanfaatan Struktur Taksonomi:** Mengintegrasikan hierarki klasifikasi biologis (famili → genus → spesies) ke dalam hierarchical loss function agar model memahami kekerabatan antar kelas, bukan hanya membedakannya secara independen.
4. **Peningkatan Inferensi:** Menerapkan Test-Time Augmentation (TTA) dan uncertainty estimation berbasis Monte Carlo Dropout untuk menghasilkan prediksi yang lebih akurat dan terkalibrasi.
5. **Pengembangan Fitur Aplikasi:** Menambahkan dukungan identifikasi berbasis kamera real-time, mode offline dengan model terkompresi, serta fitur koreksi prediksi oleh komunitas sebagai sumber data continual learning